Name: Sanskruti Dahiphale

PRN: 123B1B007

Assignment No. 4

Title:

You are on a treasure hunt and have a map that shows various caves connected by tunnels. Each tunnel has a different cost to cross, representing difficulty or danger levels. You need to calculate the safest path between each pair of caves (i.e., the path with the minimum danger). Use the Floyd-Warshall algorithm to determine the safest paths between all pairs of caves, and apply it to a network with at least 8 caves and randomly assigned danger levels for each tunnel.

Objectives:

- To apply Dynamic Programming to solve complex, overlapping subproblems.
- To implement an All-Pairs Shortest Path algorithm.

Course Outcome Mapping:

CO3 - Apply computational techniques to solve optimization and decision-making problems using either Dynamic programming or Branch and Bound.

Theory:

Dynamic Programming is an algorithmic technique used to efficiently solve complex problems by breaking them into smaller, manageable subproblems and storing their results to avoid redundant computations.

It is particularly useful when a problem exhibits the following properties:

- **Optimal Substructure:** The overall optimal solution can be constructed from the optimal solutions of its smaller subproblems.
- **Overlapping Subproblems:** The same subproblems are solved repeatedly, leading to unnecessary computations if not optimized.

In Dynamic Programming, solutions to subproblems are stored (usually in a table, array, or matrix), and each value is computed using previously solved results. This approach significantly reduces time complexity compared to recursive methods by ensuring each subproblem is solved only once.

Floyd–Warshall Algorithm:

The Floyd–Warshall Algorithm is a **Dynamic Programming (DP)** technique used to determine the shortest paths between all pairs of vertices in a weighted graph.

It works by systematically considering each vertex as an intermediate point and updating the shortest distances between every pair of vertices.

In the context of the given problem:

- **Caves** are represented as vertices (nodes).
- **Tunnels** are represented as edges connecting the caves.
- **Danger level** is treated as the weight of each edge.

The main objective is to calculate the minimum danger level (i.e., shortest path) between every pair of caves by evaluating all possible paths and selecting the safest (least dangerous) route.

Working Principle:

The algorithm improves the solution step-by-step by considering each node as an intermediate node.

For every pair of nodes (i, j), it checks whether passing through an intermediate node k gives a shorter (safer) path.

If yes, the distance is updated.

Formula Used: dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

Explanation of Formula:

• dist[i][j] → current shortest distance from i to j

• dist[i][k] + dist[k][j] → distance via intermediate node k

• We take the minimum of both values

<u>Example</u>:

Consider 4 caves (0, 1, 2, 3) with the following initial danger levels:

Initial Distance Matrix:

	0	**1**	**2**	**3**

0	0	5	∞	10
1	∞	0	3	∞
2	∞	∞	0	1
3	∞	∞	∞	0

Step 1: Using Node 0 as Intermediate

No major updates occur since no shorter paths are found through node

0. Step 2: Using Node 1 as Intermediate

Check paths through node 1:

- Path 0 → 1 → 2 = 5 + 3 = 8
  Update: dist[0][2] = 8

Updated matrix:

	0	**1**	**2**	**3**
0	0	5	8	10
1	∞	0	3	∞
2	∞	∞	0	1
3	∞	∞	∞	0

Step 3: Using Node 2 as Intermediate

Check paths through node 2:

- Path 0 → 2 → 3 = 8 + 1 = 9
  Update: dist[0][3] = 9

- Path 1 → 2 → 3 = 3 + 1 = 4
  Update: dist[1][3] = 4

Updated matrix:

	0	**1**	**2**	**3**
0	0	5	8	9
1	∞	0	3	4
2	∞	∞	0	1

3	∞	∞	∞	0

Step 4: Using Node 3 as Intermediate

No further improvements occur.

Result:

The matrix now represents the shortest (safest) paths between all pairs of caves. Key Observation:

- Initially only direct paths were known
- After applying Floyd-Warshall, indirect paths gave better (safer) results
- The algorithm ensures optimal paths between every pair

Time Complexity:

- The algorithm uses three nested loops.

  Therefore, the time complexity is $O(V^3)$

- Where V = number of vertices (caves)

This makes it suitable for dense graphs where edges are many.

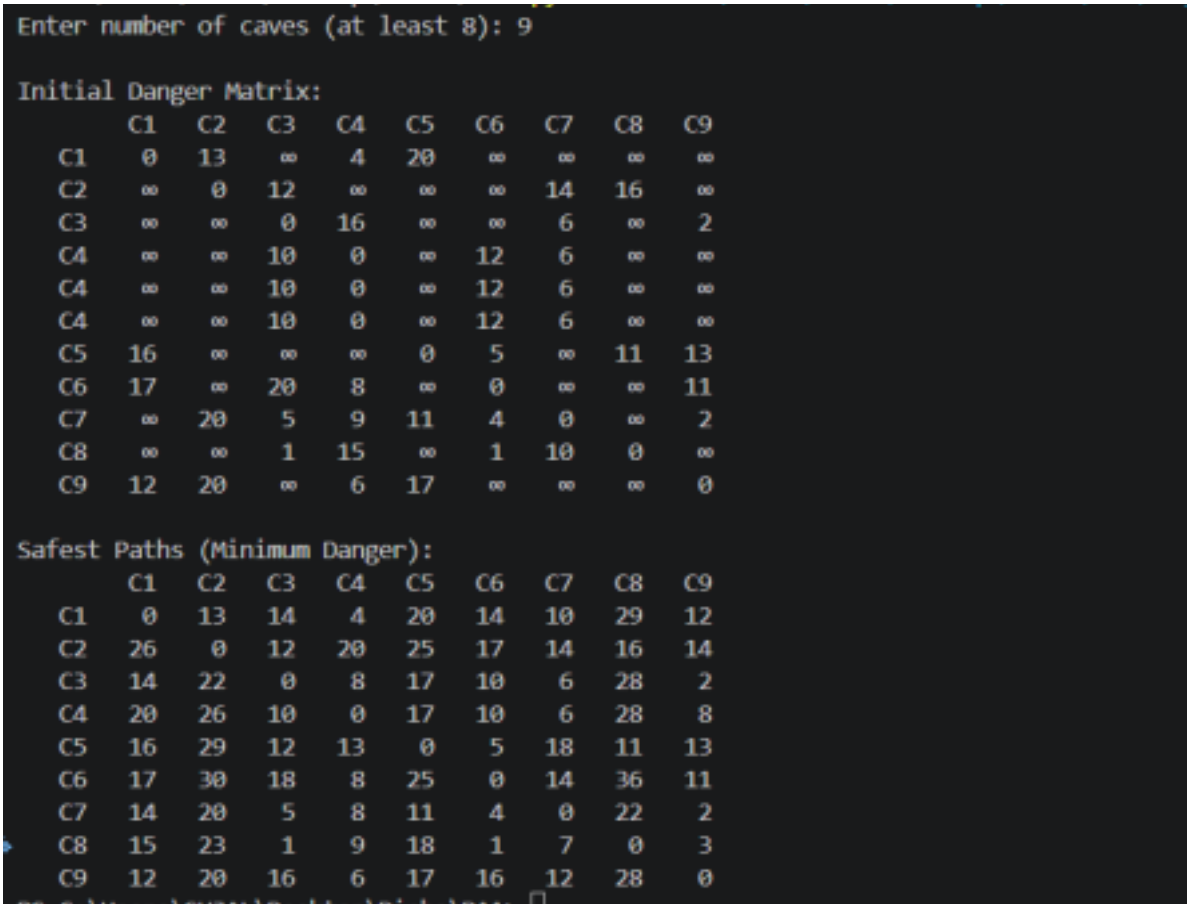
Code:

```
import random
INF = float('inf')
# input
while True:
    n = int(input("Enter number of caves (at least 8): "))
    if n >= 8:
        break
    else:
        print("Please enter a number >= 8")

# create graph
dist = []
```

```
for i in range(n):
    row = []
    for j in range(n):
        if i == j:
            row.append(0)
        else:
            if random.choice([True, False]):
                row.append(random.randint(1, 20))
            else:
                row.append(INF)
    dist.append(row)
# improved print function
def print_matrix(matrix, title):
    print("\n" + title)

    # header row
    print("     ", end="")
    for j in range(n):
        print(f"{'C'+str(j+1):>5}", end="")
    print()

    # rows
    for i in range(n):
        print(f"{'C'+str(i+1):>5}", end="")
        for j in range(n):
            if matrix[i][j] == INF:
                print(f"{'∞':>5}", end="")
            else:
                print(f"{matrix[i][j]:>5}", end="")
        print()
```

```
# print initial
print_matrix(dist, "Initial Danger Matrix:")

# Floyd-Warshall
for k in range(n):
    for i in range(n):
        for j in range(n):
            if dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k] + dist[k][j]

# print result
print_matrix(dist, "Safest Paths (Minimum Danger):")
```

Output:



Conclusion:

The Floyd-Warshall algorithm efficiently computes the shortest paths between all pairs of nodes using Dynamic Programming. It is especially useful for dense graphs where multiple paths exist between nodes.

With a time, complexity of $O(V^3)$, it is not suitable for very large graphs but works effectively for moderate-sized networks like the cave system in this problem. The algorithm guarantees the safest path (minimum danger) between every pair of caves, making it highly useful in navigation and network optimization problems.